SPPU DATA STRUCTURES PRACTICAL VIVA QUESTION BANK

Comprehensive practical-wise oral questions with exam-ready answers

COURSE	Data Structures Laboratory - SE AI & DS
PATTERN	SPPU 2024 Pattern - PCC-204-AID reference
COVERAGE	Practicals 01 to 09
INCLUDES	Core theory, algorithm logic, code questions, edge cases, complexity and quick revision

Prepared from the practical files and the organised source-code folder supplied in this conversation. The questions are probable viva prompts, not a guarantee of the exact questions an examiner will ask.

Study tip: understand the answer and be ready to point to the corresponding function or variable in your program.

How to Use This Book

This question bank is organised in the same order as your practical folders. Each practical begins with a short explanation you can say when the examiner asks, 'Explain your practical.' The questions then move from basic definitions to algorithms, implementation details, complexity and likely follow-up questions.

Contents

General DSA and programming fundamentals

Before Practical 1

Practical 01 - Searching Algorithms

Linear Search and Binary Search

Practical 02 - Sorting Algorithms

Bubble, Selection, Insertion and Shell Sort

Practical 03 - Telephone Directory Using Hashing

Linear Probing, Quadratic Probing and Double Hashing

Practical 04 - Set Operations Using Singly Linked Lists

Intersection, union, symmetric difference and neither

Practical 05 - Cinemax Seat Booking

Doubly linked lists

Practical 06 - Balanced Parentheses

Stack

Practical 07 - Pizza Parlour

Circular queue

Practical 08 - Binary Search Tree

BST operations

Practical 09 - Graph Connectivity

Adjacency matrix, DFS and BFS

Last-minute revision

Complexity table, terms and examiner checklist

Important: Read the source-alignment notes before the practical sections. Some uploaded originals differ from the corrected source files in your GitHub folder. The examiner may ask you about the program you actually compile and demonstrate.

General DSA and Programming Fundamentals

These questions can be asked before, during or after any practical.

Q. What is Data Structures?

Answer: Data Structures is the study of ways to organise and store data so that operations such as access, insertion, deletion and searching can be performed efficiently.

Q. What are linear and non-linear data structures?

Answer: In a linear data structure, elements are arranged in a sequence, as in arrays, linked lists, stacks and queues. In a non-linear structure, elements form hierarchical or network relationships, as in trees and graphs.

Q. What is an Abstract Data Type (ADT)?

Answer: An ADT defines a data type in terms of the values it stores and the operations it supports, without requiring a particular implementation. A stack ADT, for example, supports push and pop.

Q. What is an algorithm?

Answer: An algorithm is a finite, ordered set of clear steps used to solve a problem or complete a computation.

Q. What is time complexity?

Answer: Time complexity describes how the number of basic operations grows as the input size grows. It is commonly expressed using Big-O notation.

Q. What is space complexity?

Answer: Space complexity describes how memory requirements grow with input size, including the extra memory used by the algorithm.

Q. What does Big-O notation represent?

Answer: Big-O describes an asymptotic upper bound on growth. For example, O(n) grows linearly and O(n^2) grows quadratically with input size.

Q. What is a pointer?

Answer: A pointer is a variable that stores the memory address of another object or variable.

Q. What is dynamic memory allocation?

Answer: It is the process of requesting memory while a program is running. In C, malloc/free are commonly used; in C++, new/delete are commonly used.

Q. What is recursion?

Answer: Recursion is a technique in which a function calls itself to solve a smaller instance of a problem. It must have a base case to stop the calls.

Q. What is the difference between C and C++?

Answer: C is a procedural programming language. C++ supports procedural programming as well as object-oriented features such as classes, objects and encapsulation.

Q. What is a compiler?

Answer: A compiler translates source code into machine code or an intermediate executable form and reports errors that prevent compilation.

Q. What is the difference between a compile-time error and a run-time error?

Answer: A compile-time error is detected while translating the source code, often due to syntax or type problems. A run-time error occurs while the compiled program is executing.

Q. Why should a program validate user input?

Answer: Validation checks whether the supplied values are within expected ranges and formats. It helps prevent invalid array indices, incorrect results and unexpected program behaviour.

PRACTICAL 01 - Searching Algorithms

Language / source files: Linear Search: C++; Binary Search: C

This practical stores student roll numbers and checks whether a selected roll number is present. The uploaded practical separates the programs: Linear Search works on data in any order, while Binary Search is supplied sorted data.

A good 30-second answer

In this practical, I implemented two searching techniques for student roll numbers. Linear Search scans each element until the key is found, so it works with unsorted data. Binary Search repeatedly compares the key with the middle element and discards half of the search interval, so the input must be sorted. The function returns the zero-based index when found and -1 otherwise.

Oral / Viva Questions and Answers

Q. What is searching?

Answer: Searching is the process of locating a particular element, called the key, in a collection and determining whether it is present. A search may also return the element's position.

Q. What is Linear Search?

Answer: Linear Search checks array elements one by one, beginning at index 0. It stops when the key is found or when all n elements have been checked.

Q. Does Linear Search require sorted data?

Answer: No. It works with sorted or unsorted data because it examines elements sequentially.

Q. What is Binary Search?

Answer: Binary Search compares the key with the middle element of a sorted array. If the key is smaller, it continues in the left half; if larger, it continues in the right half.

Q. Why must the array be sorted for Binary Search?

Answer: The algorithm decides which half can contain the key based on the ordering. Without sorted data, that decision is not valid and the algorithm may miss an existing value.

Q. Why is Binary Search faster for large sorted arrays?

Answer: Each comparison approximately halves the remaining search range. The number of comparisons therefore grows logarithmically rather than linearly.

Q. What are low, high and mid?

Answer: low is the first index in the current search range, high is the last index, and mid is the middle index calculated from them.

Q. Why does the code calculate mid as low + (high - low) / 2?

Answer: It computes the middle index while reducing the risk of integer overflow that can occur with (low + high) / 2 when index values are very large.

Q. What does the condition low <= high mean?

Answer: It means at least one candidate index remains. When low becomes greater than high, the search interval is empty and the key is not present.

Q. What does returning -1 mean?

Answer: The key was not found. A valid array index is non-negative, so -1 is used as a not-found marker.

Q. Why does the program print position + 1?

Answer: C and C++ arrays use zero-based indexing, but people usually count positions starting from 1. Adding 1 converts the internal index into a human-readable position.

Q. What are the best, average and worst time complexities of Linear Search?

Answer: Best case is O(1), when the first element matches. Average and worst cases are O(n), because a large part or all of the array may need to be checked.

Q. What is the time complexity of Binary Search?

Answer: Best case is O(1), when the middle element is the key. Average and worst cases are O(log n), because the search range is halved after each comparison.

Q. What is the space complexity of these implementations?

Answer: Both supplied implementations are iterative and use a constant amount of extra memory, so their auxiliary space complexity is O(1). The input array itself requires O(n) storage.

Q. What happens if the array contains duplicate roll numbers?

Answer: The search returns one matching occurrence, usually the first one encountered by Linear Search or the matching middle occurrence found by Binary Search. It does not guarantee the first or last duplicate.

Q. Which language files are provided for Practical 1?

Answer: The Linear Search program is saved as linear_search.cpp and the Binary Search program is saved as binary_search.c. They are compiled separately with a C++ and a C compiler, respectively.

PRACTICAL 02 - Sorting Algorithms

Language / source files: C++

This practical sorts first-year student percentages in ascending order and displays up to five highest scores. The prepared code includes Bubble, Selection, Insertion and Shell Sort; the assignment specifically names Bubble Sort and Selection Sort.

A good 30-second answer

In this practical, I store student percentages as floating-point values and sort them in ascending order. Bubble Sort swaps adjacent out-of-order values. Selection Sort selects the smallest value from the unsorted part. The program also includes Insertion Sort and Shell Sort. After sorting, the last values are displayed in reverse order to show the top five scores from highest to lowest.

Oral / Viva Questions and Answers

Q. What is sorting?

Answer: Sorting is the process of arranging data in a particular order, such as ascending or descending order.

Q. What is Bubble Sort?

Answer: Bubble Sort repeatedly compares adjacent elements and swaps them when they are in the wrong order. After each pass, a large element moves toward its correct position near the end.

Q. Why is it called Bubble Sort?

Answer: Because larger or smaller values appear to move, or 'bubble', toward one end of the array through repeated adjacent swaps.

Q. What is Selection Sort?

Answer: Selection Sort divides the array into sorted and unsorted portions. It finds the minimum element in the unsorted portion and places it at the beginning of that portion.

Q. What is Insertion Sort?

Answer: Insertion Sort takes one element at a time and inserts it into the correct position within the already-sorted portion to its left.

Q. What is Shell Sort?

Answer: Shell Sort is an improvement over Insertion Sort that first compares elements separated by a gap. The gap is reduced over multiple passes until it becomes 1.

Q. What is a pass in a sorting algorithm?

Answer: A pass is one complete iteration of the algorithm's main comparison-and-update process over a portion of the data.

Q. What is the worst-case time complexity of Bubble Sort?

Answer: O(n^2). It may compare adjacent elements through roughly n passes, with a linear number of comparisons per pass.

Q. What is the time complexity of Selection Sort?

Answer: Best, average and worst cases are O(n^2), because it searches the remaining unsorted portion for the minimum on every pass.

Q. What is the best-case time complexity of the prepared Bubble Sort?

Answer: O(n), because the implementation uses a swapped flag. If a complete pass makes no swaps, it stops early and the input is already sorted.

Q. Which of these algorithms are stable in their usual versions?

Answer: Bubble Sort and Insertion Sort are stable when equal elements are not unnecessarily moved past each other. The usual Selection Sort and Shell Sort implementations are not stable.

Q. What does stable sorting mean?

Answer: A stable sorting algorithm preserves the original relative order of records that have equal keys.

Q. Why is the percentage array stored using float?

Answer: Student percentages can contain decimal parts, such as 78.5. Using int would discard the fractional component during storage or assignment.

Q. Why should the temporary variable in Bubble Sort also be float?

Answer: The temporary variable stores a percentage during swapping. If it were int, decimal values could be truncated and the data would be changed incorrectly.

Q. How does the program display the top five scores?

Answer: The array is first sorted in ascending order. The program then reads from the last index backward and displays up to five values, so the highest score appears first.

Q. What if there are fewer than five students?

Answer: The program displays only the available number of scores. Its loop checks both the array boundary and the count of scores already displayed.

Q. What is the extra space complexity of these in-place sorts?

Answer: The sorting routines use only a few variables for indices, keys or swaps, so their auxiliary space is O(1), apart from the vector holding the input values.

Q. Which sort is often suitable for a small or nearly sorted array?

Answer: Insertion Sort is often suitable because it has low overhead and can run in O(n) time when the array is already or nearly sorted.

PRACTICAL 03 - Telephone Directory Using Hashing

Language / source files: C++

This practical stores names and telephone numbers in a hash table. The prepared program lets the user select Linear Probing, Quadratic Probing or Double Hashing and supports create, display, search, update and delete operations.

A good 30-second answer

In this practical, I implemented a telephone directory using a hash table. A hash function converts a client's name into an index. When two names map to the same index, a collision occurs. I handle collisions using the selected probing method: Linear Probing, Quadratic Probing or Double Hashing. The program supports all basic record operations.

Oral / Viva Questions and Answers

Q. What is hashing?

Answer: Hashing is a technique that maps a key to an index in a hash table using a hash function. It is used to support efficient insertion, searching and deletion.

Q. What is a hash table?

Answer: A hash table is an array-like data structure in which records are stored at positions calculated by a hash function.

Q. What is a hash function?

Answer: A hash function takes a key and calculates a table index. In this program, the character codes of the name are added and the sum is reduced modulo the table size.

Q. What is a collision?

Answer: A collision occurs when two different keys produce the same initial hash index.

Q. What is collision resolution?

Answer: Collision resolution is the method used to find another available table position when the position calculated by the hash function is already occupied.

Q. Explain Linear Probing.

Answer: Linear Probing checks consecutive positions. Its probe formula is (h(k) + i) % m, where i is the attempt number and m is the table size.

Q. Explain Quadratic Probing.

Answer: Quadratic Probing checks positions using a quadratic offset, such as (h(k) + i*i) % m. The step distances increase quadratically rather than by one.

Q. Explain Double Hashing.

Answer: Double Hashing uses a second hash function to calculate the step size. Its probe formula is (h1(k) + i*h2(k)) % m.

Q. What is clustering in hashing?

Answer: Clustering is the formation of groups of occupied positions that can increase probe lengths. Linear Probing is especially affected by primary clustering.

Q. What is the load factor?

Answer: The load factor is the number of occupied records divided by the table capacity. A higher load factor generally increases the chance of collisions and longer searches.

Q. Why is the table size 101 in this program?

Answer: 101 is a prime number. A prime table size can help distribute probe sequences and is particularly useful when choosing a suitable step size for Double Hashing.

Q. Why does the program use the same probing method for search as for insertion?

Answer: A record is stored at a position reached through a particular probe sequence. Search must follow that same sequence to find the record reliably.

Q. What is a tombstone?

Answer: A tombstone is a marker that says a record was deleted but the slot must not be treated as never-used. It allows a later search to continue through the probe chain.

Q. Why not simply mark a deleted slot as empty?

Answer: If a colliding record was inserted farther along the probe sequence, search might stop at the newly empty slot and incorrectly report that the later record is absent.

Q. What do status values 0, 1 and 2 represent?

Answer: In this implementation, 0 means empty, 1 means occupied and 2 means deleted (tombstone).

Q. What is the average and worst-case time complexity of hashing?

Answer: With a low load factor and a good distribution, operations are expected to take O(1) time. In the worst case, many collisions can make an operation O(n).

Q. What is the time complexity of displaying the directory?

Answer: O(m), where m is the table capacity, because the program scans the table to print all occupied records.

Q. Can two different names have the same character-sum hash?

Answer: Yes. Different strings can have the same sum of character codes, so the hash is not unique. Collision resolution is therefore still required.

Q. What happens if a record with an existing name is inserted again?

Answer: The prepared program updates the existing record's telephone number instead of storing a duplicate name.

PRACTICAL 04 - Set Operations Using Singly Linked Lists

Language / source files: C++

This practical stores the roll numbers of students who like Vanilla and Butterscotch ice cream in two singly linked lists. It computes intersection, symmetric difference, union and the number who like neither flavour.

A good 30-second answer

In this practical, I create two singly linked lists. List A contains students who like Vanilla and list B contains students who like Butterscotch. By traversing the lists and checking membership, I calculate the intersection, symmetric difference and union. I calculate the number who like neither flavour by subtracting the union size from the total class strength.

Oral / Viva Questions and Answers

Q. What is a linked list?

Answer: A linked list is a dynamic linear data structure made of nodes. Each node stores data and one or more links to other nodes.

Q. What does a node in a singly linked list contain?

Answer: It contains the data field, which stores the roll number here, and a next pointer that stores the address of the following node.

Q. What is the head pointer?

Answer: The head pointer stores the address of the first node in a linked list. If the list is empty, head is null.

Q. What does NULL or nullptr indicate?

Answer: It indicates that a pointer does not point to a valid node. In a singly linked list, the last node's next pointer is null.

Q. What is the difference between an array and a linked list?

Answer: An array stores elements in contiguous memory and supports direct indexing. A linked list stores nodes dynamically, does not require contiguous memory, and is traversed through pointers.

Q. What is a set in mathematics?

Answer: A set is a collection of distinct elements. Duplicate elements are not counted more than once.

Q. What is the intersection of A and B?

Answer: A intersection B contains the elements that appear in both sets. In this practical, these are students who like both flavours.

Q. What is the union of A and B?

Answer: A union B contains every distinct element that appears in A or B or both. It represents students who like at least one of the two flavours.

Q. What is the symmetric difference?

Answer: The symmetric difference contains elements that belong to exactly one set, but not to both. It is the union after excluding the intersection.

Q. How is the number who like neither flavour calculated?

Answer: Neither = total class strength - number of distinct students in the union of A and B.

Q. Why does the program avoid duplicate roll numbers?

Answer: A set must contain distinct elements. The appendUnique function checks whether a roll number already exists before creating another node.

Q. What does the contains function do?

Answer: It traverses a linked list from its head and returns true if the requested roll number is found; otherwise it returns false after reaching the end.

Q. What does appendUnique do?

Answer: It first checks whether the value already exists. If not, it creates a new node and attaches it at the end of the list.

Q. Why is dynamic memory allocation used?

Answer: The number of students is known at runtime. The new operator creates nodes as needed rather than requiring a fixed-size array of nodes.

Q. Why does the program call delete when it finishes?

Answer: It releases dynamically allocated nodes and prevents memory leaks. The freeList function deletes nodes one by one and moves the head forward.

Q. What is the time complexity of checking membership in a singly linked list?

Answer: O(n), because in the worst case each node must be examined. Checking many elements across two lists can therefore take O(a*b) time for list sizes a and b.

Q. Why can neither become negative?

Answer: It should not be negative if the entered total class strength is consistent. The program warns if the union is larger than the stated class strength and clamps the reported value to zero.

PRACTICAL 05 - Doubly Linked List - Cinemax Seat Booking

Language / source files: C++

This practical manages a cinema with 10 rows and 7 seats per row. The prepared program maintains an array of 10 head pointers, with one doubly linked list for each row, and supports seat display, booking, available-seat listing and cancellation.

A good 30-second answer

In this practical, I implemented a Cinemax seat-booking system using doubly linked lists. The theatre has 10 rows and 7 seats per row. An array stores the head pointer for each row. Every seat node stores the seat number, booking status, customer ID, previous pointer and next pointer. A booking is accepted only if the seat is valid and available, and cancellation requires the correct customer ID.

Oral / Viva Questions and Answers

Q. What is a doubly linked list?

Answer: It is a linked list in which each node stores a pointer to the next node and a pointer to the previous node, allowing traversal in both directions.

Q. What fields are stored in each seat node?

Answer: The prepared code stores the seat number, booking status, customer ID, prev pointer and next pointer.

Q. What is the purpose of the prev pointer?

Answer: It points to the previous node in the row, enabling backward traversal and easier navigation to the preceding seat.

Q. What is the purpose of the next pointer?

Answer: It points to the next node in the row and is used to move forward through the seats.

Q. What is the difference between a singly and a doubly linked list?

Answer: A singly linked list has only a next pointer and supports forward traversal. A doubly linked list has both prev and next pointers and supports forward and backward traversal.

Q. Why is an array of head pointers used?

Answer: There are 10 theatre rows. The array keeps one head pointer for each row, so each row has its own separate doubly linked list of seven seat nodes.

Q. How many total seats are represented?

Answer: 10 rows multiplied by 7 seats per row gives 70 seats.

Q. What does a head pointer contain?

Answer: It contains the address of the first seat node in a row. In an empty row it would be nullptr.

Q. How is the first node linked?

Answer: The first node's prev pointer is nullptr because there is no preceding node. Its next pointer refers to the second node, if one exists.

Q. How is the last node linked?

Answer: The last node's next pointer is nullptr because there is no following node. Its prev pointer refers to the preceding seat.

Q. How does the program book a seat?

Answer: It validates the row and seat ranges, finds the node, checks that it is not already booked, reads the customer ID, marks it booked and stores the ID.

Q. How does the program cancel a booking?

Answer: It validates the row and seat, checks that the seat is booked, asks for the customer ID and compares it with the stored ID. It clears the booking only when the IDs match.

Q. Why is customer ID checked during cancellation?

Answer: It prevents a different user from cancelling a booking that belongs to someone else.

Q. What are the initial pre-bookings in the prepared sample?

Answer: Row 1 seat 3 and row 4 seat 6 are marked as booked initially, to demonstrate occupied seats.

Q. Is the prepared program's list circular?

Answer: No. It creates a normal, non-circular doubly linked list for each row. The first node's prev and last node's next are nullptr.

Q. What is the complexity of finding a seat in one row?

Answer: O(7) in this fixed-size example, because the program may traverse up to seven nodes. Updating a node after it is found takes O(1).

Q. What is the important difference between the uploaded document and this prepared source?

Answer: The uploaded PDF shows one circular doubly linked list containing all 70 seats. The prepared GitHub version follows the written requirement more closely by using an array of row head pointers and a separate non-circular list for each row.

PRACTICAL 06 - Stack - Balanced Parentheses

Language / source files: C++

This practical checks whether delimiters in an expression are balanced. The prepared implementation uses an array-based stack and checks the correct matching pairs of parentheses, square brackets and braces.

A good 30-second answer

In this practical, I use a stack to check whether an expression is well-parenthesized. A stack follows LIFO. I scan the expression from left to right and push each opening delimiter. When a closing delimiter appears, I verify that the stack is not empty and that its top element is the matching opening delimiter. At the end, the expression is balanced only if the stack is empty.

Oral / Viva Questions and Answers

Q. What is a stack?

Answer: A stack is a linear data structure that follows the Last In, First Out principle. The last element inserted is the first element removed.

Q. What is LIFO?

Answer: LIFO stands for Last In, First Out. It describes the order in which elements are removed from a stack.

Q. What is push?

Answer: Push inserts a new element at the top of the stack.

Q. What is pop?

Answer: Pop removes and returns the top element of the stack.

Q. What is peek or top?

Answer: Peek or top returns the current top element without removing it.

Q. What does top = -1 mean?

Answer: It represents an empty array-based stack. The first push increments top to zero and stores the first element at index zero.

Q. What is stack underflow?

Answer: Underflow occurs when a program tries to pop or access the top of an empty stack.

Q. What is stack overflow?

Answer: Overflow occurs when a program tries to push an element into a fixed-size stack that is already full.

Q. Why is a stack suitable for checking parentheses?

Answer: The most recently opened delimiter must be closed first. This is exactly the LIFO behaviour provided by a stack.

Q. How does the algorithm process an opening delimiter?

Answer: When it encounters (, [or {, it pushes that character onto the stack.

Q. How does it process a closing delimiter?

Answer: It first checks that the stack is not empty, then checks whether the top opening delimiter matches the closing delimiter. If it matches, it pops the top.

Q. Why is checking only whether the stack is empty not enough?

Answer: An expression such as ([)] has the same number of opening and closing delimiters, but the types are mismatched. The code must verify the exact pair at the top.

Q. Is the expression ([)] balanced?

Answer: No. The closing parenthesis does not match the most recent opening square bracket. It is rejected by the matching-pair check.

Q. What condition is checked at the end?

Answer: The stack must be empty. If any opening delimiter remains, it was never closed and the expression is invalid.

Q. What is the time complexity?

Answer: O(n), where n is the number of characters, because the algorithm scans each character once and each push or pop takes O(1) time.

Q. What is the worst-case auxiliary space complexity?

Answer: O(n), because an expression containing n opening delimiters may require storing all of them in the stack.

Q. What happens to letters and arithmetic operators?

Answer: They are ignored by this delimiter-checking algorithm. It checks delimiter balance only; it does not evaluate the expression or validate its arithmetic syntax.

Q. Why does the prepared program use getline?

Answer: getline reads the full line, including spaces, so an expression containing spaces can be checked as one string.

PRACTICAL 07 - Queue - Pizza Parlour

Language / source files: C++

This practical simulates a pizza parlour accepting a limited number of orders and serving them in first-come, first-served order. The prepared program uses a circular array queue with a count variable and can store all five positions.

A good 30-second answer

In this practical, I implemented a circular queue for a pizza parlour. A queue follows FIFO, so the first order accepted is the first order served. The addOrder function inserts at the rear, while serveOrder removes from the front. Circular indexing using the modulo operator reuses array positions after orders are served.

Oral / Viva Questions and Answers

Q. What is a queue?

Answer: A queue is a linear data structure that follows First In, First Out. The first element inserted is the first element removed.

Q. What is FIFO?

Answer: FIFO means First In, First Out. It describes the order in which elements leave a queue.

Q. What is enqueue?

Answer: Enqueue is the operation of inserting a new element at the rear of the queue.

Q. What is dequeue?

Answer: Dequeue is the operation of removing the element at the front of the queue.

Q. Why is a queue suitable for the pizza parlour?

Answer: Orders should normally be served in the same order in which they were accepted. FIFO queue behaviour models this first-come, first-served rule.

Q. What is a circular queue?

Answer: A circular queue treats the last array position as connected to the first position. When the rear reaches the end, it can wrap around to index zero if space is available.

Q. Why use a circular queue instead of a simple linear array queue?

Answer: A circular queue reuses positions freed by dequeue operations. A simple linear queue may leave unused positions at the beginning that cannot be reused without shifting elements.

Q. What does the modulo operator do in rearIndex = (rearIndex + 1) % MAX_ORDERS?

Answer: The modulo operation wraps the index back to zero after it reaches the last array position, keeping the index within valid bounds.

Q. What do frontIndex and rearIndex represent?

Answer: frontIndex identifies the next order to be served. rearIndex identifies the most recently inserted order.

Q. What is the purpose of the count variable?

Answer: It stores the number of currently pending orders. It distinguishes a full queue from an empty queue and allows all five array positions to be used.

Q. How does the program detect a full queue?

Answer: It checks whether count equals MAX_ORDERS. If count is five, no more orders can be accepted until an order is served.

Q. How does the program detect an empty queue?

Answer: It checks whether count is zero. A serve request on an empty queue is rejected as underflow.

Q. What are the initial values of the queue variables?

Answer: The constructor initializes frontIndex to 0, rearIndex to -1 and count to 0. This represents an empty queue before any order is accepted.

Q. How many orders can the prepared program store?

Answer: MAX_ORDERS is 5 and the count variable tracks occupancy, so all five array positions can be used.

Q. What is the time complexity of enqueue and dequeue?

Answer: Both are O(1), because each operation updates a small number of indices and variables without shifting the other elements.

Q. What is the time complexity of displaying pending orders?

Answer: O(k), where k is the number of current pending orders, because the program prints each order once.

Q. What is the difference between a stack and a queue?

Answer: A stack follows LIFO and inserts/removes at the top. A queue follows FIFO, inserts at the rear and removes from the front.

PRACTICAL 08 - Binary Search Tree Operations

Language / source files: C

This practical constructs a BST and performs insertion, longest-path calculation, minimum-value lookup, mirror swapping, searching and inorder traversal. The source tracks whether the tree has been mirrored so that insert, minimum and search use the correct ordering direction afterward.

A good 30-second answer

In this practical, I construct a Binary Search Tree by inserting each value according to the BST property: smaller values go to the left and larger values go to the right. I also calculate the height, find the minimum value, mirror the tree by swapping left and right pointers, search for a value and display inorder traversal. The code tracks the mirrored state because the ordering direction reverses after mirroring.

Oral / Viva Questions and Answers

Q. What is a tree data structure?

Answer: A tree is a non-linear hierarchical data structure made of nodes connected by edges. It has a root node and subtrees.

Q. What is a binary tree?

Answer: A binary tree is a tree in which each node has at most two children, commonly called the left child and the right child.

Q. What is a Binary Search Tree?

Answer: A BST is a binary tree in which, for a normal BST, values in the left subtree are smaller than the node and values in the right subtree are larger than the node.

Q. What is the root node?

Answer: The root is the topmost node of a tree. It has no parent.

Q. What is a leaf node?

Answer: A leaf node is a node that has no children; both its left and right pointers are null.

Q. What is the purpose of the left and right pointers?

Answer: They store links to the roots of the left and right subtrees of a node.

Q. How is a value inserted into a normal BST?

Answer: Compare the value with the current node. If it is smaller, continue into the left subtree; if larger, continue into the right subtree. Create the new node at the first null position.

Q. What happens to duplicate values in this prepared program?

Answer: Duplicate values are ignored. The insertion function only creates a node when it reaches a null pointer and does not insert a value equal to an existing node.

Q. What is tree traversal?

Answer: Traversal is the process of visiting every node in a defined order. Common depth-first traversals are inorder, preorder and postorder.

Q. What is inorder traversal?

Answer: Inorder traversal visits the left subtree, then the root node, then the right subtree. In a normal BST, it prints values in ascending order.

Q. What is the height of the tree in this program?

Answer: The program measures height in number of nodes on the longest root-to-leaf path. An empty tree has height zero; a single-node tree has height one.

Q. How is the longest path calculated?

Answer: The height function recursively calculates the left and right subtree heights and returns 1 plus the larger of the two.

Q. How do you find the minimum value in a normal BST?

Answer: Follow the left pointer repeatedly until there is no left child. That leftmost node contains the minimum value.

Q. What is a skewed tree?

Answer: A skewed tree is a tree in which nodes mostly have only one child, making the structure similar to a linked list.

Q. What is the average and worst-case search complexity of a BST?

Answer: For a reasonably balanced tree, search is O(log n) on average. For a highly skewed tree, search can become O(n).

Q. What does mirroring a tree mean?

Answer: Mirroring means swapping the left and right child pointers at every node. The resulting tree is the mirror image of the original.

Q. Why does normal BST search need a change after mirroring?

Answer: Mirroring reverses the ordering direction. A smaller value is now found along the right side rather than the left, so comparison directions must be reversed. The prepared code uses a mirrored flag for this.

Q. What is the time complexity of height calculation and mirroring?

Answer: Both are O(n), because each node must be visited once.

Q. What is dynamic memory allocation in this C program?

Answer: Each node is created using malloc, which requests memory at runtime. The program later releases all allocated nodes using free.

PRACTICAL 09 - Graph Connectivity Using DFS and BFS

Language / source files: C++

This practical models cities as vertices and flight paths as weighted edges. The prepared program stores the graph in an adjacency matrix, performs DFS and BFS, and checks whether every city is reachable from the selected start vertex.

A good 30-second answer

In this practical, I represent each city as a vertex and each direct flight as an edge. I use an adjacency matrix to store the graph, where a non-zero value represents a direct flight and zero means no direct flight. I perform DFS using recursion and BFS using a queue. Then I check whether all vertices were visited to determine whether the undirected graph is connected.

Oral / Viva Questions and Answers

Q. What is a graph?

Answer: A graph is a non-linear data structure consisting of vertices and edges. It can represent relationships or connections between objects.

Q. What do vertices and edges represent in this practical?

Answer: Vertices represent cities or airports. Edges represent direct flight paths between pairs of cities.

Q. What is a weighted graph?

Answer: A weighted graph is a graph in which each edge stores a value such as distance, travel time, cost or fuel consumption.

Q. What is an adjacency matrix?

Answer: An adjacency matrix is a two-dimensional V by V array. The entry at row i and column j indicates whether an edge exists between vertices i and j and can store the edge weight.

Q. What does zero mean in this program's adjacency matrix?

Answer: Zero means that there is no direct flight between that pair of cities. A non-zero value represents an edge and its stored cost, time or fuel value.

Q. What is the difference between an adjacency matrix and adjacency list?

Answer: An adjacency matrix uses O(V^2) space and provides a direct edge lookup. An adjacency list stores only each vertex's neighbours and is generally more space-efficient for sparse graphs.

Q. Is the prepared graph directed or undirected?

Answer: It is undirected. The program stores each entered route value in both adj[i][j] and adj[j][i], so a route works in both directions.

Q. How would a directed graph be represented?

Answer: For a directed edge from i to j, set adj[i][j] but do not automatically set adj[j][i]. The reverse edge would need to be entered separately.

Q. What is DFS?

Answer: Depth First Search explores one unvisited neighbour as far as possible before backtracking. This implementation uses recursion.

Q. What is BFS?

Answer: Breadth First Search visits vertices level by level, exploring immediate neighbours before moving to the next level. This implementation uses a queue.

Q. Which data structure is used by DFS here?

Answer: The DFS function is recursive, so the program uses the system call stack to remember the vertices to return to during backtracking.

Q. Which data structure is used by BFS?

Answer: BFS uses a queue. Vertices are inserted at the rear and removed from the front in FIFO order.

Q. Why is a visited array required?

Answer: It prevents a vertex from being processed repeatedly, especially when the graph contains cycles, and ensures the traversal terminates.

Q. When is an undirected graph connected?

Answer: An undirected graph is connected when there is a path between every pair of vertices. Starting from any one vertex, a traversal should therefore visit all vertices.

Q. How does the program check connectivity?

Answer: It starts a traversal from the selected vertex using a fresh visited vector. If any vertex remains unvisited, the graph is disconnected; otherwise it is connected.

Q. What is the time complexity of DFS or BFS using an adjacency matrix?

Answer: O(V^2), because for each visited vertex the program scans its full row of V possible neighbours.

Q. What is the space complexity of an adjacency matrix?

Answer: O(V^2), because it stores V by V entries. The visited vector and traversal queue or recursion stack require up to O(V) additional space.

Q. What is the difference between connected and complete graphs?

Answer: A connected graph has a path between every pair of vertices. A complete graph has a direct edge between every pair of distinct vertices. Every complete graph with at least one vertex is connected, but not every connected graph is complete.

Q. Does the program's DFS output always have to match its BFS output?

Answer: No. DFS and BFS use different visiting strategies, so their traversal orders may differ even though they reach the same connected component.

Q. What is a connected component?

Answer: A connected component is a maximal set of vertices in which every vertex is reachable from every other vertex within that set.

Important Source-Alignment Notes

The question bank follows the organised source files in your SPPU-DSA-Practicals folder, while also taking account of the original documents you uploaded. Be clear about which version you are demonstrating.

Practical 1 - Searching

The supplied folder has separate files: linear_search.cpp (C++) and binary_search.c (C). Binary Search validates that the roll numbers are entered in ascending order.

Practical 2 - Sorting

The assignment statement names Bubble Sort and Selection Sort. The uploaded program also contained Insertion Sort and Shell Sort; the organised sorting.cpp includes all four and adds top-five display. The temporary variable for percentages is float.

Practical 3 - Hashing

The original uploaded DOCX implements Linear Probing only. The organised telephone_directory.cpp adds Linear Probing, Quadratic Probing and Double Hashing, uses a tombstone for deletion and fixes the deletion-index issue.

Practical 4 - Set Operations

The uploaded PDF builds linked lists and transfers their values into the C++ STL set container for calculations. The organised set_operations.cpp performs intersection, symmetric difference and union directly through linked-list traversal. Explain the code version you are showing.

Practical 5 - Doubly Linked List

The uploaded PDF shows one circular doubly linked list for all 70 seats. The organised cinemax_booking.cpp uses an array of 10 row-head pointers, with a separate non-circular doubly linked list of 7 seats per row, matching the written array-of-head-pointers requirement.

Practical 6 - Stack

The original uploaded program pops for each closing delimiter without verifying the matching bracket type and can pop an empty stack. The organised parenthesis_checker.cpp checks underflow and matching pairs for (), [] and {}.

Practical 7 - Queue

The uploaded queue reserves one slot to distinguish full from empty, so MAX=5 provides only four usable positions. The organised pizza_parlour.cpp uses a count variable and can store all five orders; it also removes the Windows-only Sleep dependency.

Practical 8 - BST

The organised binary_search_tree.c tracks whether the tree is mirrored. After swapping child pointers, it reverses the comparison direction for insert, minimum and search. The simple original search routine does not account for that reversed ordering.

Practical 9 - Graph Connectivity

The uploaded DOCX uses a symmetric adjacency matrix and displays DFS and BFS traversals, but does not explicitly report connectivity. The organised graph_connectivity.cpp adds the all-vertices-reached check. Because values are copied to both matrix directions, this implementation models an undirected graph.

Last-Minute Complexity Revision

Memorise the main time and space costs. If the examiner asks for a complexity, mention the input size and the reason briefly.

Practical / operation	Time complexity	Key reason
P1 Linear Search	Best O(1); avg/worst O(n)	May inspect each array element.
P1 Binary Search	Best O(1); avg/worst O(log n)	Halves the sorted search range.
P2 Bubble Sort	Best O(n) with early stop; worst O(n^2)	Adjacent comparisons over passes.
P2 Selection Sort	O(n^2) in best/avg/worst	Scans remaining unsorted values each pass.
P2 Insertion Sort	Best O(n); worst O(n^2)	Shifts elements into sorted prefix.
P2 Shell Sort	Depends on gap sequence	Prepared simple halving-gap method; worst commonly O(n^2).
P3 Hash table operations	Expected O(1); worst O(n)	Depends on load factor and collisions.
P3 Display table	O(m)	Scans all m table positions.
P4 Linked-list membership	O(n)	May traverse the full list.
P4 Set comparisons	O(a*b) for pairwise membership checks	Each element in one list may scan the other.
P5 Seat search per row	O(7), fixed-size example	At most seven nodes in a row.
P6 Balanced parentheses	O(n) time; O(n) space	Each character is processed once; stack stores openers.
P7 Enqueue / dequeue	O(1)	Only indices and count are updated.
P7 Display queue	O(k)	Prints k pending orders.
P8 BST insert/search/min	O(h); average O(log n), worst O(n)	Follows one root-to-leaf path.
P8 Height/mirror/inorder	O(n)	Visits every tree node.
P9 DFS/BFS with matrix	O(V^2)	Scans V matrix entries for each visited vertex.
P9 Matrix storage	O(V^2) space	Stores V rows by V columns.

Quick definitions

LIFO: Last In, First Out - stack.

FIFO: First In, First Out - queue.

Collision: Two different keys map to the same hash-table index.

Load factor: Number of occupied hash entries divided by table capacity.

Tombstone: Deleted-slot marker that preserves the hash probe chain.

Visited: Marker showing that a graph vertex has already been discovered/processed.

Root / leaf: Top node of a tree / node with no children.

Intersection / union: Common elements / all distinct elements across sets.

Practical Oral - Final Checklist

Before your viva, make sure you can do each of the following without reading directly from the screen:

☐ State the aim and describe the real-world problem in two or three sentences.

☐ Identify the main data structure used and explain why it is suitable.

☐ Explain the main function and the most important variables in your code.

☐ Trace one small example manually, showing the important index, pointer, stack, queue or traversal changes.

☐ State the time complexity and explain what causes it.

☐ Explain one edge case: not found, empty, full, duplicate, invalid input or disconnected graph.

☐ Compile and run the exact source file you plan to show the examiner.

☐ Do not claim a feature exists unless it is present in the source file you are demonstrating.

Useful compile commands

C++ source: **g++ -std=c++17 filename.cpp -o program.exe**
Run in Windows PowerShell: **.\program.exe**
C source: **gcc filename.c -o program.exe**
Run in Windows PowerShell: **.\program.exe**

Final reminder

An examiner may ask a different question, ask you to trace your program, or request a small modification. This guide covers the core concepts and likely follow-ups for the nine practicals in your organised folder, but it cannot predict the exact oral questions. Read the source-alignment notes so your explanation matches the code you actually run.

End of Viva Question Bank